LIMBAJE DE DESCRIERE HARDWARE

TEMA 12

PSPICE:DFFTR

VHDL+VERILOG: DFFTR SI NUMĂRĂTOR BINAR REVERSIBIL

TEHNOLOGIA CMOS IMPUSA: CMOS_TSMC_025_T51V.TXT

ANUL 3 BALABAN DAVID 5318

CONF.DR.ING. DĂNUȚ BURDIA

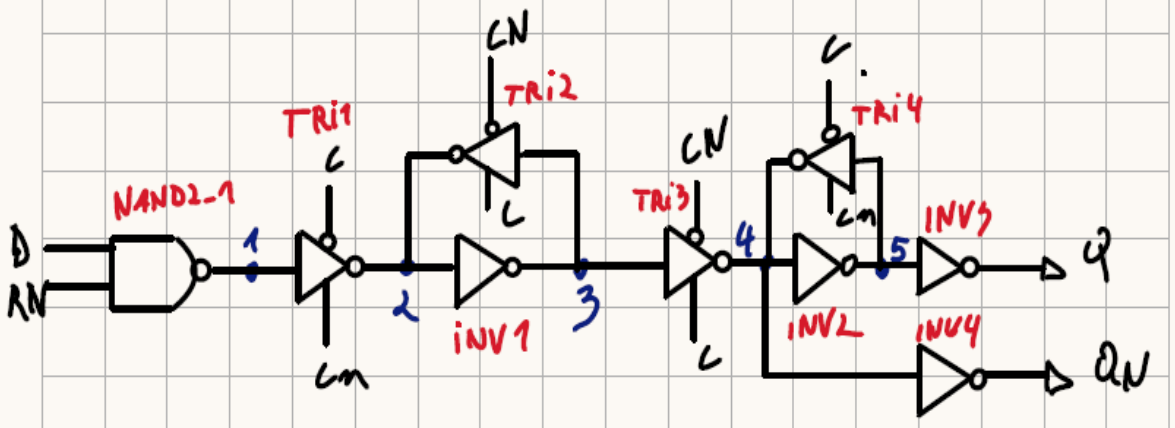
ANUL UNIVERSITAR 2025-2026

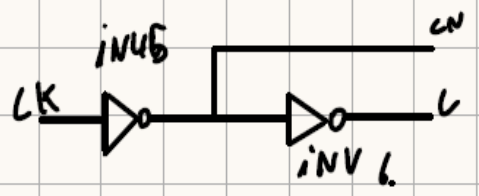
PARTEA I – PROIECTAREA ŞI ANALIZA DINAMICĂ ÎN SPICE A CIRCUITULUI DFFTR

Celula DFFTR este:

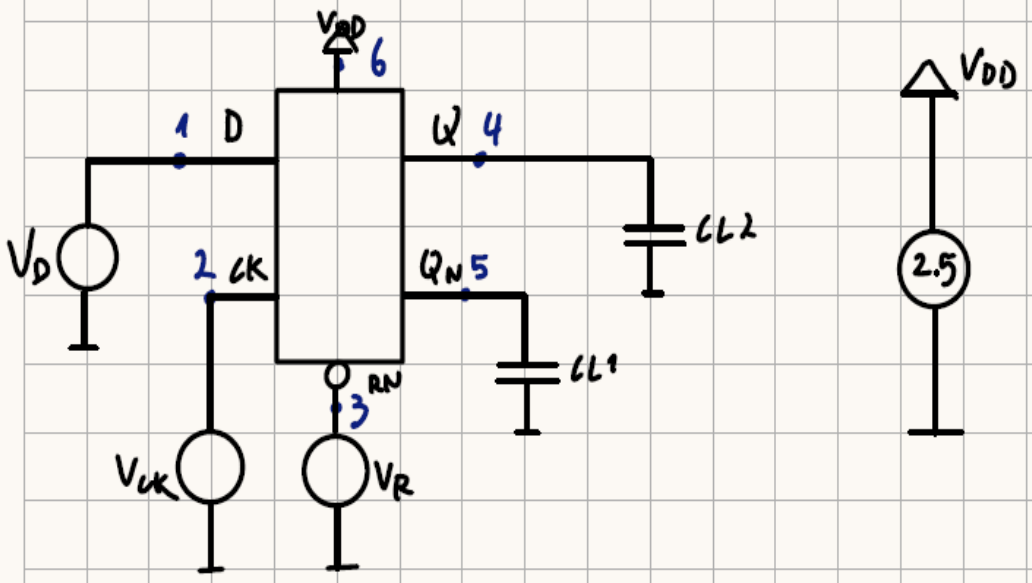
-un circuit basculant bistabil de tip D (flip-flop D)
-static
-declanșat pe frontul crescător (pozitiv)
- cu resetare sincronă activă pe nivel jos (RN).

1. Să se dimensioneze tranzistoarele MOS corespunzător tehnologiei date şi să se scrie netlist-ul SPICE al circuitului. (Netlist-ul se va scrie ierarhic folosind subcircuite).



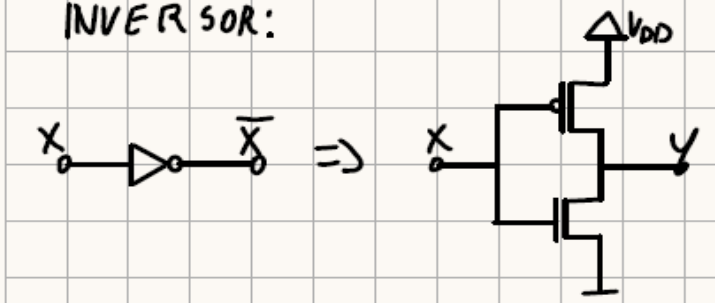


Circuit de simulare:



PORTI LOGICE CMOS: $K = \frac{306.116}{10646} = 2.88$

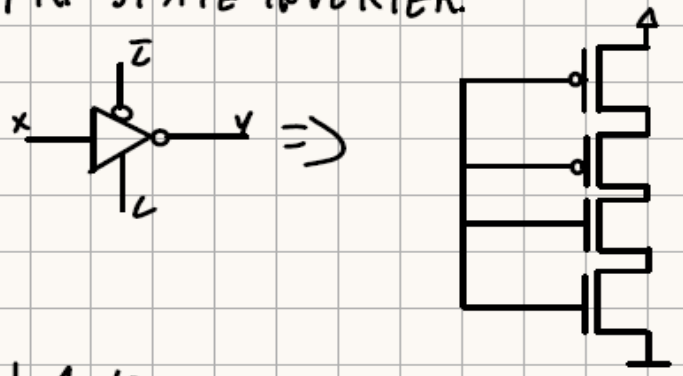
INVERSOR:



$W_P = 2.88\mu$

$W_N = 1\mu$

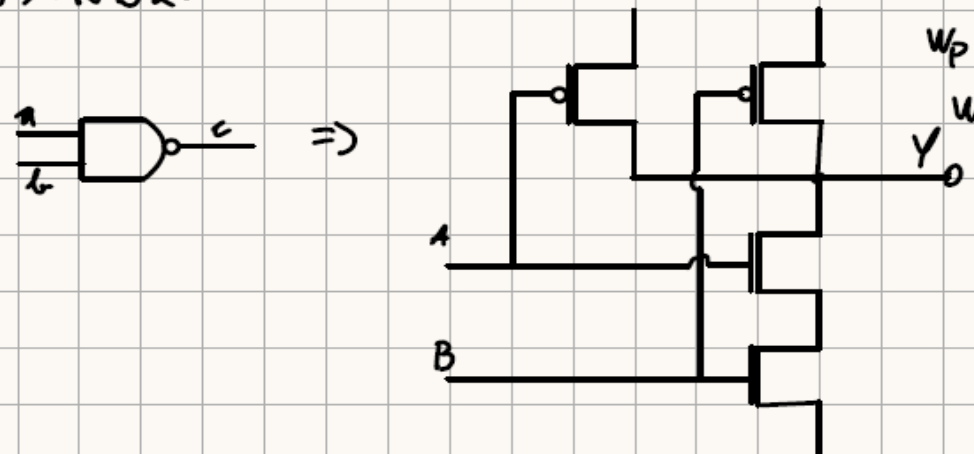
TRI-STATE INVERTER:



$W_P = 2.88\mu$

$W_N = 1\mu$

NAND2:



$W_P = 1.44\mu$

$W_n = 1\mu$

```
*********************************************
* INV IN VDD OUT
.SUBCKT INV_25 x Vdd y
MP1 y x Vdd Vdd CMOSP L=0.25u
W=2.88u
MN1 y x 0 0 CMOSN L=0.25u W=1.0u
.ENDS
*********************************************
**********
* INVTRI IN VDD OUT
* x=intrare, cn=clock_negat,
c=clock, y=iesire
.SUBCKT INVTRI_25 x cn c Vdd y
MP1 1 x Vdd Vdd CMOSP L=0.25u
W=2.88u
MP2 y cn 1 Vdd CMOSP L=0.25u
W=2.88u
MN1 y c 2 0 CMOSN L=0.25u W=1.0u
MN2 2 x 0 0 CMOSN L=0.25u W=1.0u
.ENDS
```

```
*********************************************
* NAND-2 IN VDD OUT
.SUBCKT NAND2_25 a b Vdd y
* PMOS in PARALEL
MP1 y a Vdd Vdd CMOSP L=0.25u
W=1.44u
MP2 y b Vdd Vdd CMOSP L=0.25u
W=1.44u
* NMOS in SERIE
MN1 1 a 0 0 CMOSN L=0.25u
W=1.0u
MN2 y b 1 0 CMOSN L=0.25u
W=1.0u
.ENDS
```

```
*********************************************
* DFFTR IN VDD OUT
.SUBCKT DFFTR_25 D CK RN Vdd Q
QN
XNAND2 D RN Vdd 1 NAND2_25
XTRI1 1 c cn Vdd 2 INVTRI_25
XTRI2 3 cn c Vdd 2 INVTRI_25
XTRI3 3 cn c Vdd 4 INVTRI_25
XTRI4 5 c cn Vdd 4 INVTRI_25
XINV1 2 Vdd 3 INV_25
XINV2 4 Vdd 5 INV_25
XINV3 5 Vdd Q INV_25
XINV4 4 Vdd QN INV_25
XINV5 CK Vdd cn INV_25
XINV6 cn Vdd c INV_25
.ENDS
*********************************************
*U0= 306.116512
*-------------------------------------
*U0 = 106.4695414
*=>K=2.88;
*Subcircuite PSPICE:
*********************************************
```

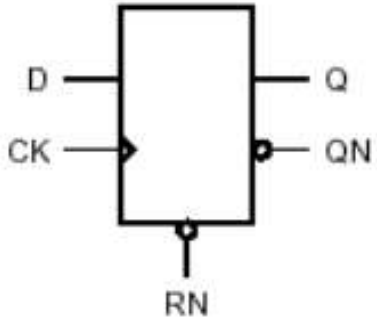
2. Să se efectueze analiza tranzitorie a circuitului pentru a pune în evidență funcționarea sa. La fiecare ieşire a circuitului se va considera cîte o capacitate de sarcină. Valorile pentru tensiunea de alimentare VDD, timpii de tranziție TR ai formelor de undă de la intrări şi, respectiv, capacitățile de sarcină CL se vor adopta în funcție de tehnologia impusă conform tabelului următor:

Tehnologie CMOS	0.13 μm	0.18μm	0.25μm	0.35μm	0.50μm
VDD	1.2V	1.8V	2.5V	3.3V	5V
TR	0.1ns	0.15ns	0.2ns	0.3ns	0.4n
CL	0.06pF	0.07pF	0.08pF	0.09pF	0.1pF

Cell Description

The DFFTR cell is a positive-edge triggered, static D-type flip-flop with synchronous active-low reset (RN).

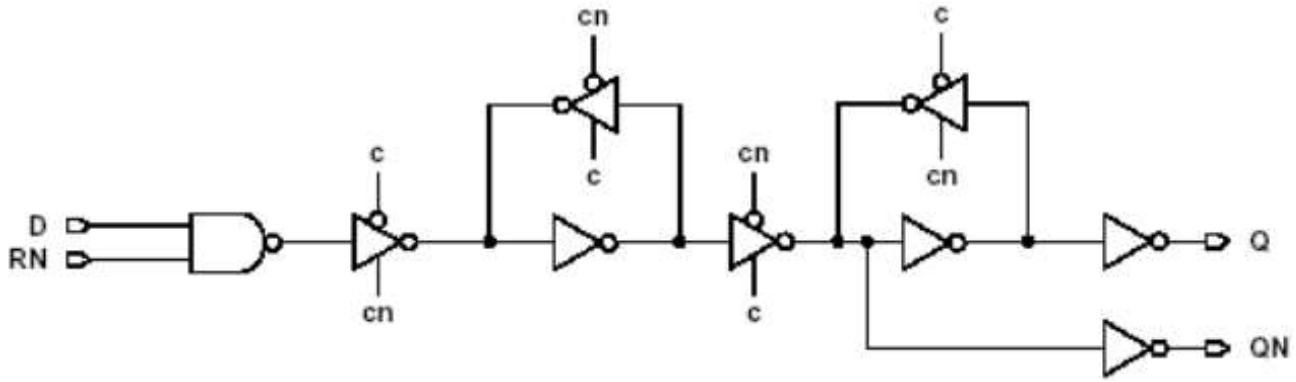
Logic Symbol

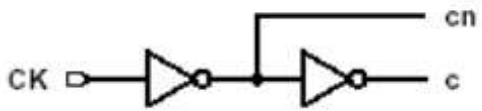


Function Table

RN	D	CK	Q[n+1]	QN[n+1]
0	x	↑	0	1
x	x	↓	Q[n]	QN[n]
1	0	↑	0	1
1	1	↑	1	0

Functional Schematic





****

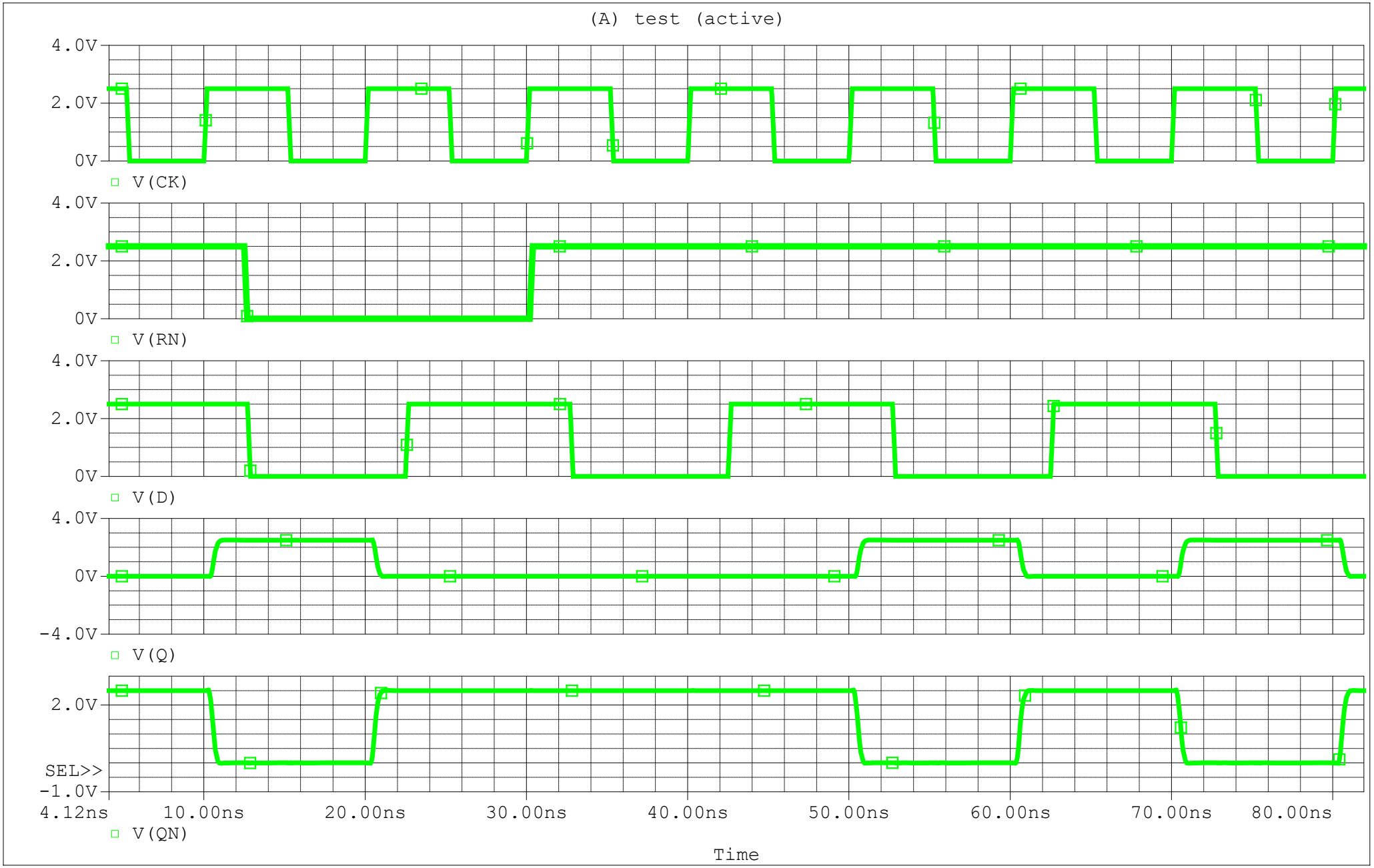
Fisier_test PSPICE:

```
*CIRCUIT DE TEST
VD D 0 0 PULSE(0 2.5 5n {TR} {TR} 5n 10n)
VCK CK 0 0 PULSE(0 2.5 0 {TR} {TR} 5n 10n)
VR RN 0 0 PULSE(2.5 0 12.5n {TR} {TR} 17.5n 1)
VDD VDD 0 2.5
XUUT D CK RN VDD Q QN DFFTR_25
CL2 Q 0 {CVAL}
CL1 QN 0 {CVAL}
.PARAM CVAL=0.08p
.PARAM TR=0.2n
.INC "CMOS_TSMC_025_T51V.txt"
.INC "subcircuite.txt"
.TRAN 100p 200n 0n
.STEP PARAM CVAL 0.045p 0.135p 0.005p
.PROBE
.END
```

****

*CIRCUIT DE TEST

Date/Time run: 01/08/26 12:59:53 Temperature: 27.0



3. Determinați prin simulare parametrică şi folosind funcții țintă în Probe dependența timpilor de propagare a circuitului (de la intrarea de clock la ieşiri) în funcție de capacitatea de sarcină (CL), respectiv în funcție de timpul de tranziție (TR) al formei de undă al intrării de clock. În ambele analize parametrice variația parametrilor se va face în intervalul (0.5 ... 1.5)* val. adoptată conform tehnologiei. Determinați şi notați pe graficele obținute valorile întârzierilor de propagare corespunzătoare valorilor adoptate pentru CL şi TR. Considerând că dependența timpilor de propagare în funcție de CL este de forma: tptotal = tpintrinsec + Kload*CL determinați pe baza graficelor obținute valorile pentru tpintrinsec şi Kload şi completați un tabel de felul următor (tpintrinsec corespunde valorii CL=0pF):

```
TPLH_HL_INV_BUF:
tpHLInv (1,2) = x2-x1
{
1| search forward
level(50%,p)!1;
2| search forward
level(50%,n)!2;
}
```

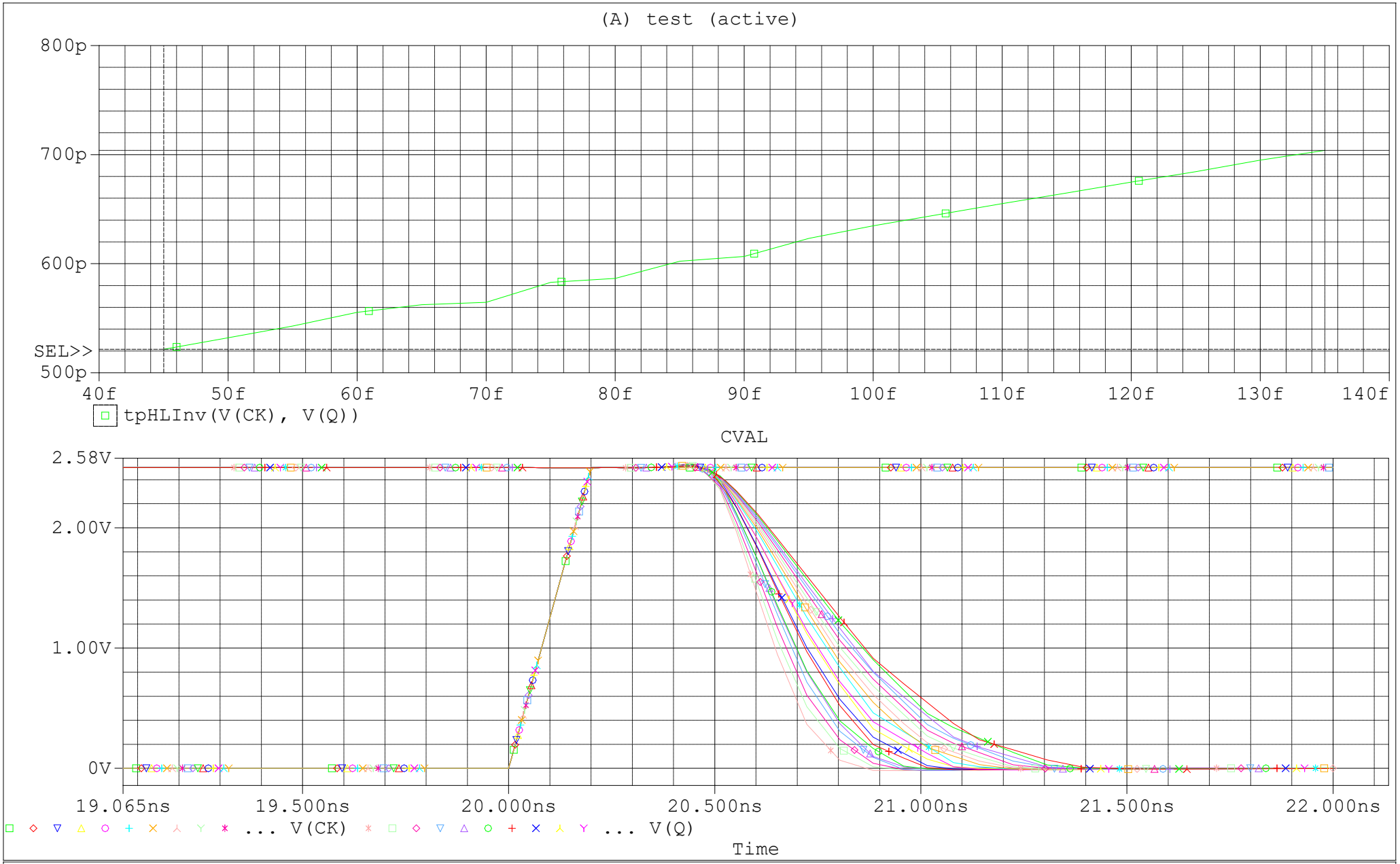
```
tpLHInv (1,2) = x2-x1
{
1| search forward
level(50%,n)!1;
2| search forward
level(50%,p)!2;
}
```

```
tpLHBuf (1,2) = x2-x1
{
1| search forward
level(50%,p)!1;
2| search forward
level(50%,p)!2;
}
```

```
tpHLBuf (1,2) = x2-x1
{
1| search forward
level(50%,n)!1;
2| search forward
level(50%,n)!2;
}
```

*CIRCUIT DE TEST

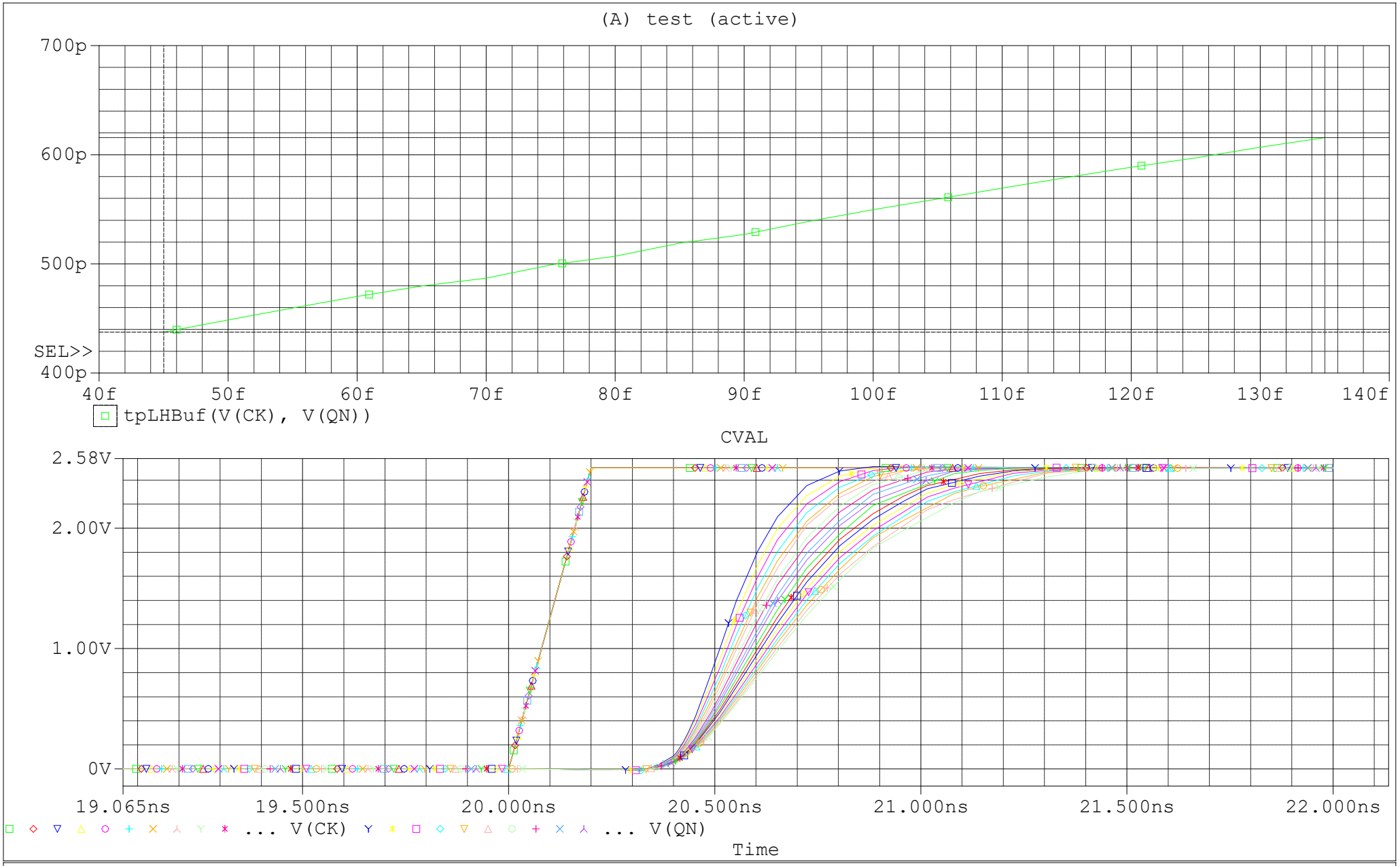
Date/Time run: 01/08/26 17:05:58 Temperature: 27.0



A1:(135.000f,703.974p) A2:(45.000f,521.574p) DIFF(A):(90.000f,182.399p)

*CIRCUIT DE TEST

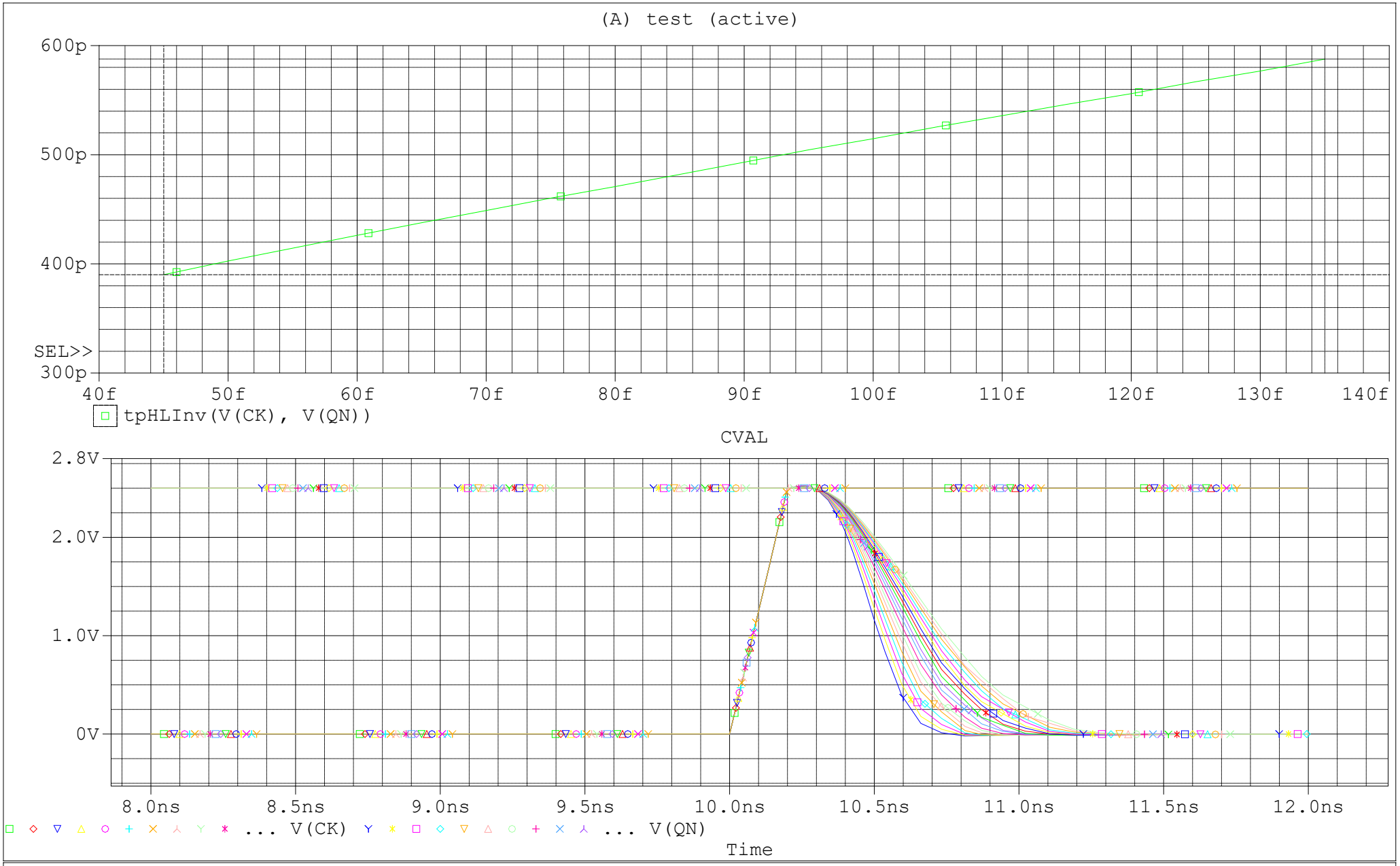
Date/Time run: 01/08/26 17:05:58 Temperature: 27.0



A1:(135.000f,615.736p) A2:(45.000f,437.533p) DIFF(A):(90.000f,178.203p)

*CIRCUIT DE TEST

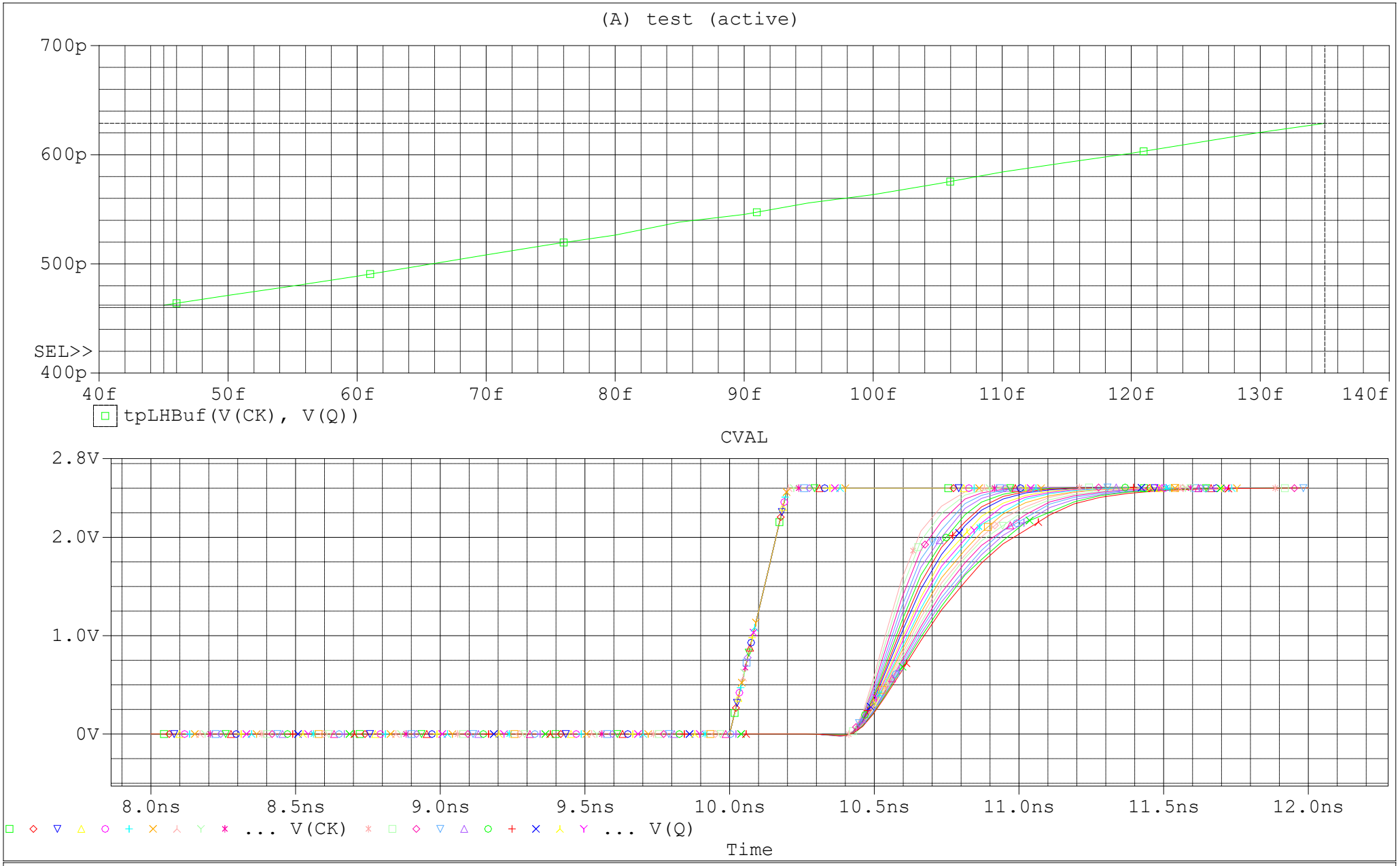
Date/Time run: 01/08/26 16:59:16 Temperature: 27.0



A1:(135.000f,587.725p) A2:(45.000f,390.018p) DIFF(A):(90.000f,197.707p)

*CIRCUIT DE TEST

Date/Time run: 01/08/26 16:59:16 Temperature: 27.0



A1:(45.000f,462.158p) A2:(135.000f,628.789p) DIFF(A):(-90.000f,-166.632p)

Masuratori timpi propagare in functie de capacitate de sarcina

Capacitate sarcina	tpLH		tpHL	
CVAL	tpLHintrinsec	Kload(s/F)	tpHLintrinsec	Kload(s/F)
CK->Q	378.862p	1851	521.634p	2026
CK->QN	348,438p	1980	291,265p	2196

Calculele:

(1,1)
A1:(45.000f,462.158p) A2:(135.000f,628.789p) DIFF(A):(90.000f,166.632p)
166.632p/90.000f

(1,3)
A1:(135.000f,703.974p) A2:(45.000f,521.574p) DIFF(A):(90.000f,182.399p)
182.399p/90.000f

(2,1)
A1:(135.000f,615.736p) A2:(45.000f,437.533p) DIFF(A):(90.000f,178.203p)
178.203p/90.000f

(2,3)
A1:(135.000f,587.725p) A2:(45.000f,390.018p) DIFF(A):(90.000f,197.707p)
197.707p/90.000f

(1,1) Tpintrinsec=timp-kload*C=462.158p-1851*45f=462.158p-83.295p=378.862p
(1,3) Tpintrinsec=timp-kload*C=703.974p-2026*90f=703.974-182,34=521.634p
(2,1) Tpintrinsec=timp-kload*C=615.736p-1980*135f=615.736p-267,300=348,438p
(2,3) Tpintrinsec=timp-kload*C=587.725p-2196*135f=587.725p-296.460=291,265p

(1,1)Tpintrinsec=timp-kload*C=462.158p-1851*45f=462.158p-83.295p=378.862p

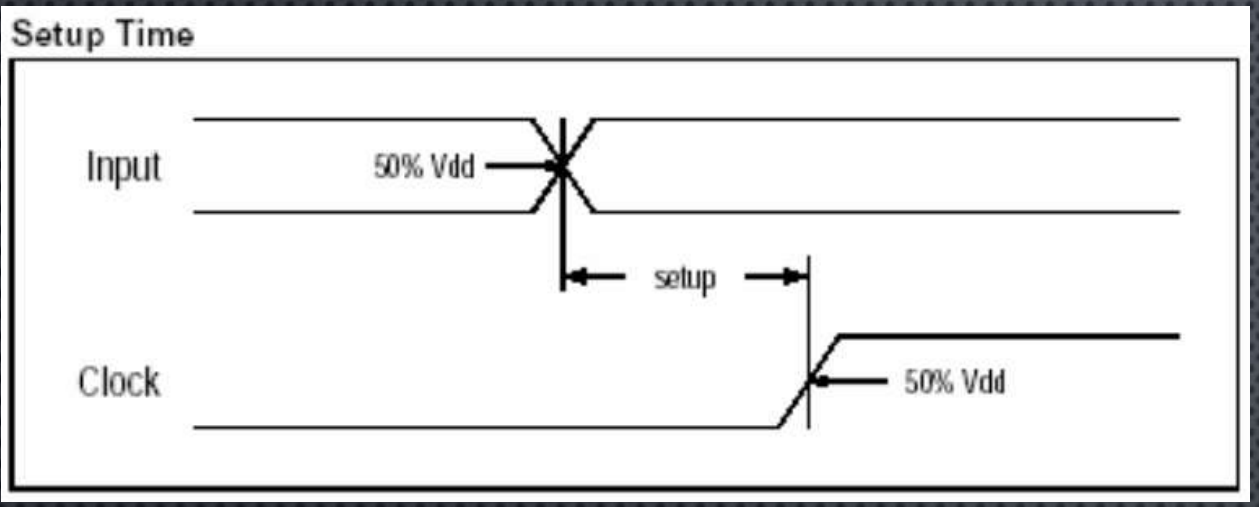
(1,3) Tpintrinsec=timp-kload*C=703.974p-2026*90f=703.974-182,34=521.634p

(2,1) Tpintrinsec=timp-kload*C=615.736p-1980*135f=615.736p-267,300=348,438p

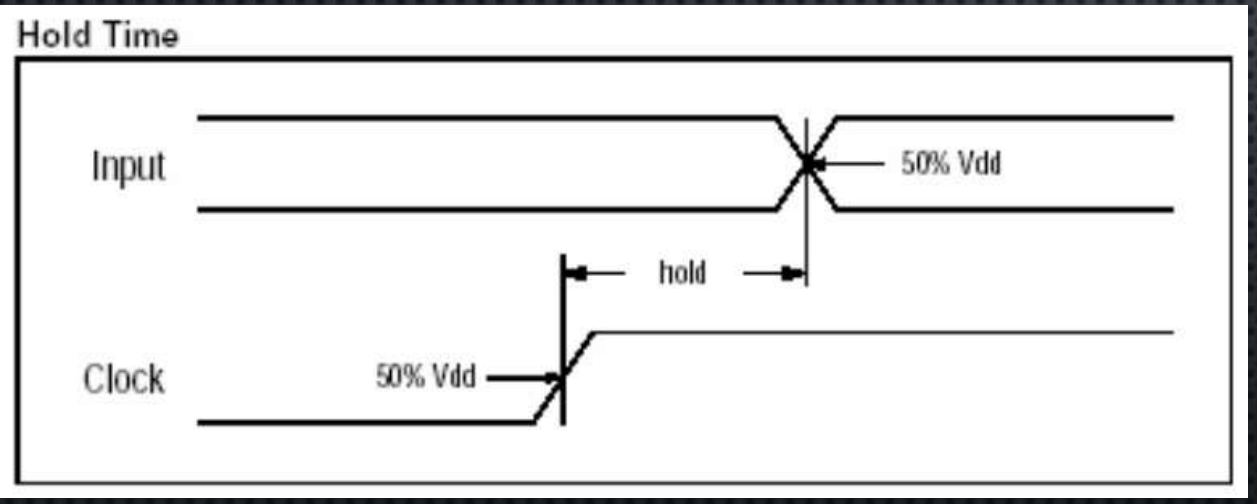
(2,3) Tpintrinsec=timp-kload*C=587.725p-2196*135f=587.725p-296.460=291,265p

4. Determinați prin simulare parametrii de constrângere SETUP_TIME, HOLD_TIME şi durata minimă a pulsului MINPW pentru pinii de intrare specificați în tabelul următor: Pentru simulările efectuate la fiecare din punctele 2, 3 şi 4 se vor prezenta schema de test, fişierul SPICE (.cir) şi formele de undă sau caracteristicile reprezentative pe baza cărora s-au determinat parametrii ceruți în tabele.

Timpul de setup se defineste ca timpul minim in care intrarile ,unui circuit secvential ,trebuie sa aiba o stare bine definita inaintea urmatorului front activ de ceas.



Timpul de mentinere se defineste ca timpul minim in care intrarile ,unui circuit secvential ,trebuie sa aiba o stare bine definita dupa frontul activ de ceas anterior.



PARTEA A II-A

VHDL-Verilog

A)

VHDL-DFFTR

A. i) Să se implementeze în VHDL un model comportamental pentru circuitul proiectat şi analizat dinamic în Partea I. În cadrul modelului se vor defini si utiliza constante generice pentru parametrii dinamici determinați prin simulare (timpi de propagare clock→ieşire, setup_time şi hold_time).

Model comportmental DFFTR VHDL

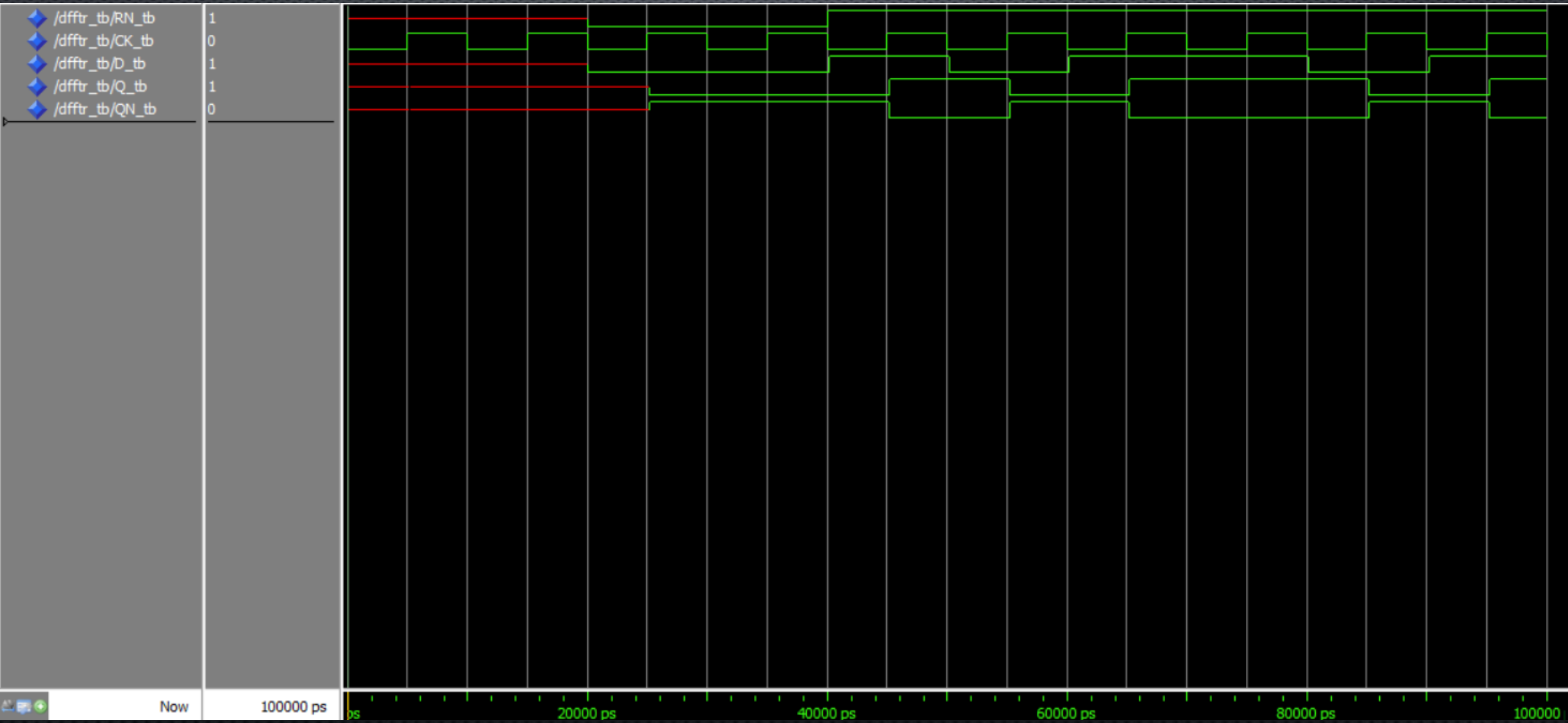
```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity dfftr is
	Generic(
		tq:time:=0.2 ns;
		tqn:time:=0.2 ns
);
	Port(
		CK:in STD_LOGIC;
		D :in STD_LOGIC;
		RN:in STD_LOGIC;
		Q :out STD_LOGIC;
		QN:out STD_LOGIC
);
end dfftr;
```

```
architecture comp of dfftr is
begin
	process(CK)
	begin
		if rising_edge(CK) then
		if RN='0' then
			Q<='0' after tq;
			QN<='1' after tqn;
		elsif RN='1' then
			Q<=D after tq;
			QN<=not D after tqn;
		else
			Q<='X' after tq;
			QN<='X' after tqn;
		end if;
end if;
end process;
end comp;
```

ii) Să se implementeze o entitate de test şi să se simuleze modelul de la punctul i)

Testbench:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity dfftr_tb is
end dfftr_tb;
architecture simulare of dfftr_tb is
    component dfftr
        Generic (
            tq:time:=0.2 ns;
            tqn:time:=0.2 ns
        );
        Port (
            CK:in STD_LOGIC;
            D:in STD_LOGIC;
            RN:in STD_LOGIC;
            Q:out STD_LOGIC;
            QN:out STD_LOGIC
        );
    end component;
    signal CK_tb:STD_LOGIC:='0';
    signal D_tb:STD_LOGIC:='0';
    signal RN_tb:STD_LOGIC:='0';
    signal Q_tb:STD_LOGIC;
    signal QN_tb:STD_LOGIC;
    constant T:time:=10 ns;
begin
    DUT:dfftr
    generic map(
        tq=>0.2 ns,
        tqn=>0.2 ns
    )
    port map(
        CK=>CK_tb,
        D=>D_tb,
        RN=>RN_tb,
        Q=>Q_tb,
        QN=>QN_tb
    );
    c_proces:process
    begin
        CK_tb<='0';
        wait for T / 2;
        CK_tb<='1';
        wait for T / 2;
    end process;
   Proces:process
    begin
RN_tb<='X';
        D_tb<='X';
        wait for 20 ns;
        RN_tb<='0';
        D_tb<='0';
        wait for 20 ns;
        RN_tb<='1';
        wait for 0.2 ns;
        D_tb<='1';
        wait for T;
        D_tb<='0';
        wait for T;
        D_tb<='1';
        wait for 2 * T;
        D_tb<='0';
        wait for T;
        D_tb<='1';
        wait for T;
        D_tb<='1';
        RN_tb<='0';
        wait for T;
        RN_tb<='1';
        D_tb<='0';
        wait for T;
        wait;
    end process;
end simulare;
```


/dfftr_tb/RN_tb
/dfftr_tb/CK_tb
/dfftr_tb/D_tb
/dfftr_tb/Q_tb
/dfftr_tb/QN_tb
1
0
1
1
0
Now
100000 ps
20000 ps
40000 ps
60000 ps
80000 ps

B)

VERILOG-DFFTR

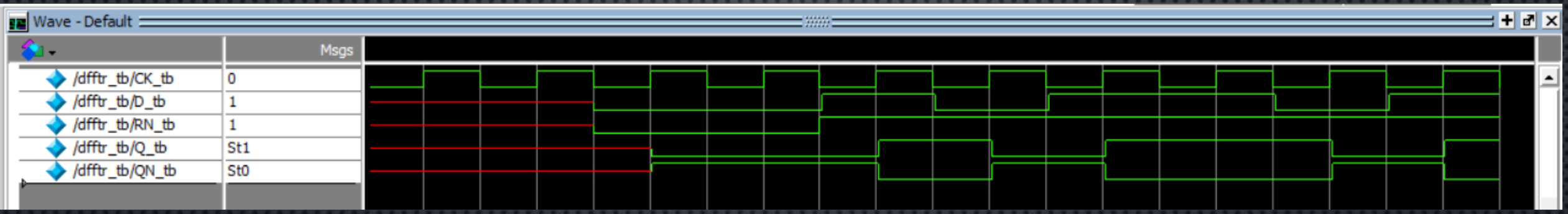
B. Să se rezolve aceleaşi cerințe de la pct. A în limbajul Verilog.

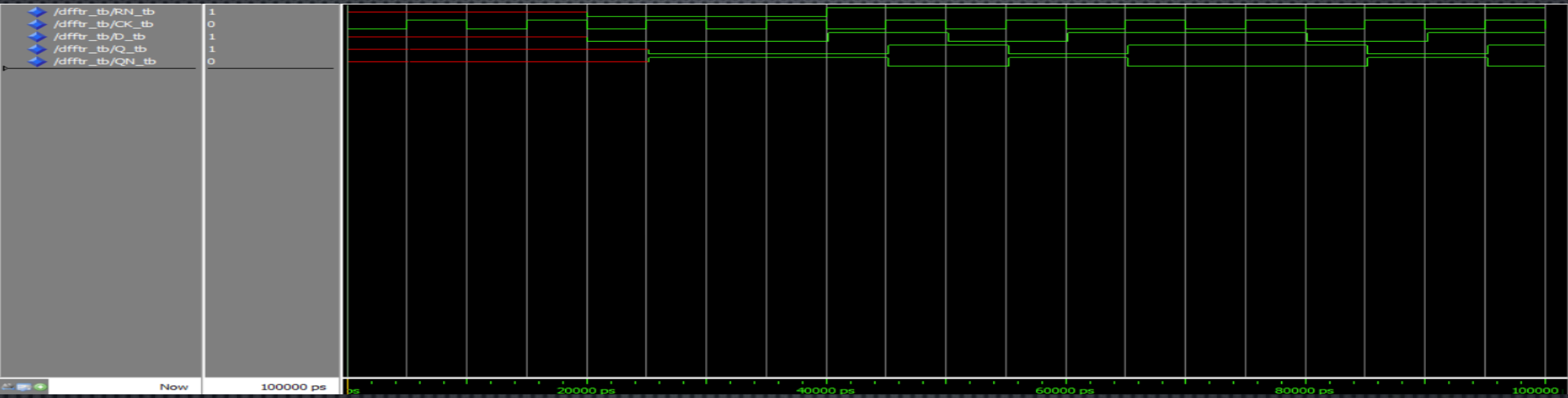
Model comportmental DFFTR Verilog

```
`timescale 1ns / 1ps

module dfftr #(

    parameter real tq =0.2,
    parameter real tqn =0.2
)(

    input wire CK,
    input wire D,
    input wire RN,
    output reg Q,
    output reg QN
);

    always @(posedge CK) begin

        if (RN==1'b0) begin
            Q<=#tq 1'b0;
            QN<=#tqn 1'b1;
        end
        else if (RN==1'b1) begin
            Q<=#tq D;
            QN<=#tqn ~D;
        end
        else begin
            Q<=#tq 1'bx;
            QN<=#tqn 1'bx;
        end
    end
endmodule
```

Testbench:

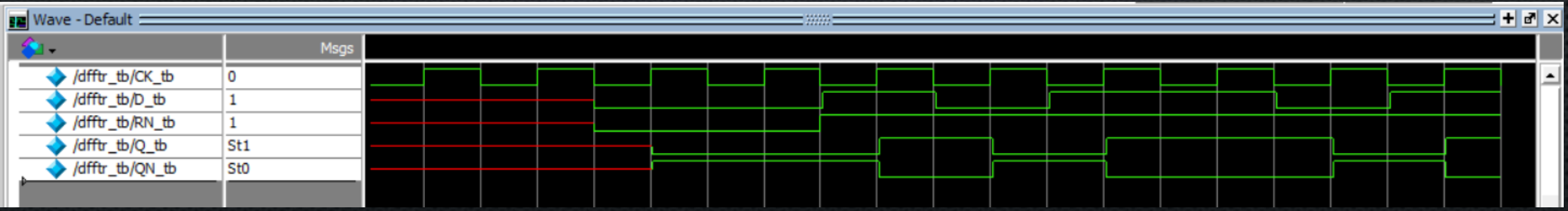
```
`timescale 1ns / 1ps

module dfftr_tb;

   reg CK_tb;
   reg D_tb;
   reg RN_tb;

   wire Q_tb;
   wire QN_tb;

   localparam CLK_PERIOD = 10;

   dfftr #(
       .tq(0.2),
       .tqn(0.2)
   ) DUT (
       .CK(CK_tb),
       .D(D_tb),
       .RN(RN_tb),
       .Q(Q_tb),
       .QN(QN_tb)
   );

   initial begin
       CK_tb = 0;
       forever begin

           #(CLK_PERIOD / 2.0) CK_tb =
~CK_tb;
       end
   end

   initial begin
       RN_tb = 1'bx;
       D_tb  = 1'bx;
       #20;

       RN_tb = 0;
       D_tb  = 0;
       #20;

       RN_tb = 1;
       #0.2;

       D_tb = 1;
       #(CLK_PERIOD);

       D_tb = 0;
       #(CLK_PERIOD);

       D_tb = 1;
       #(2 * CLK_PERIOD);

       D_tb = 0;
       #(CLK_PERIOD);

       D_tb = 1;
       #(CLK_PERIOD);

       D_tb  = 1;
       RN_tb = 0;
       #(CLK_PERIOD);

       RN_tb = 1;
       D_tb  = 0;
       #(CLK_PERIOD);

       $stop;
   end
endmodule
```


Wave - Default
Msgs
/dfftr_tb/CK_tb
0
/dfftr_tb/D_tb
1
/dfftr_tb/RN_tb
1
/dfftr_tb/Q_tb
St1
/dfftr_tb/QN_tb
St0



Putem vedea ca formele de unda sunt corespunzatoare si similare la ambele limbaje de descriere hardware

VHDL
VS
Verilog



C.NUMĂRĂTOR BINAR REVERSIBIL

Descriere: Numărătorul cu încărcare paralelă numără până când valoarea de la ieşire este egală cu limita de numărare de la portul de intrare count_to. Circuitul funcționează pe frontul crescător al semnalului de ceas

Cerințe:

- modelarea comportamentală în VHDL a circuitului.
- realizarea unui program de test (testbench) pentru simularea şi verificarea modelului circuitului de mai sus. Testbench-ul va fi realizat astfel încât să fie testate diversele ipostaze din funcționarea circuitului.

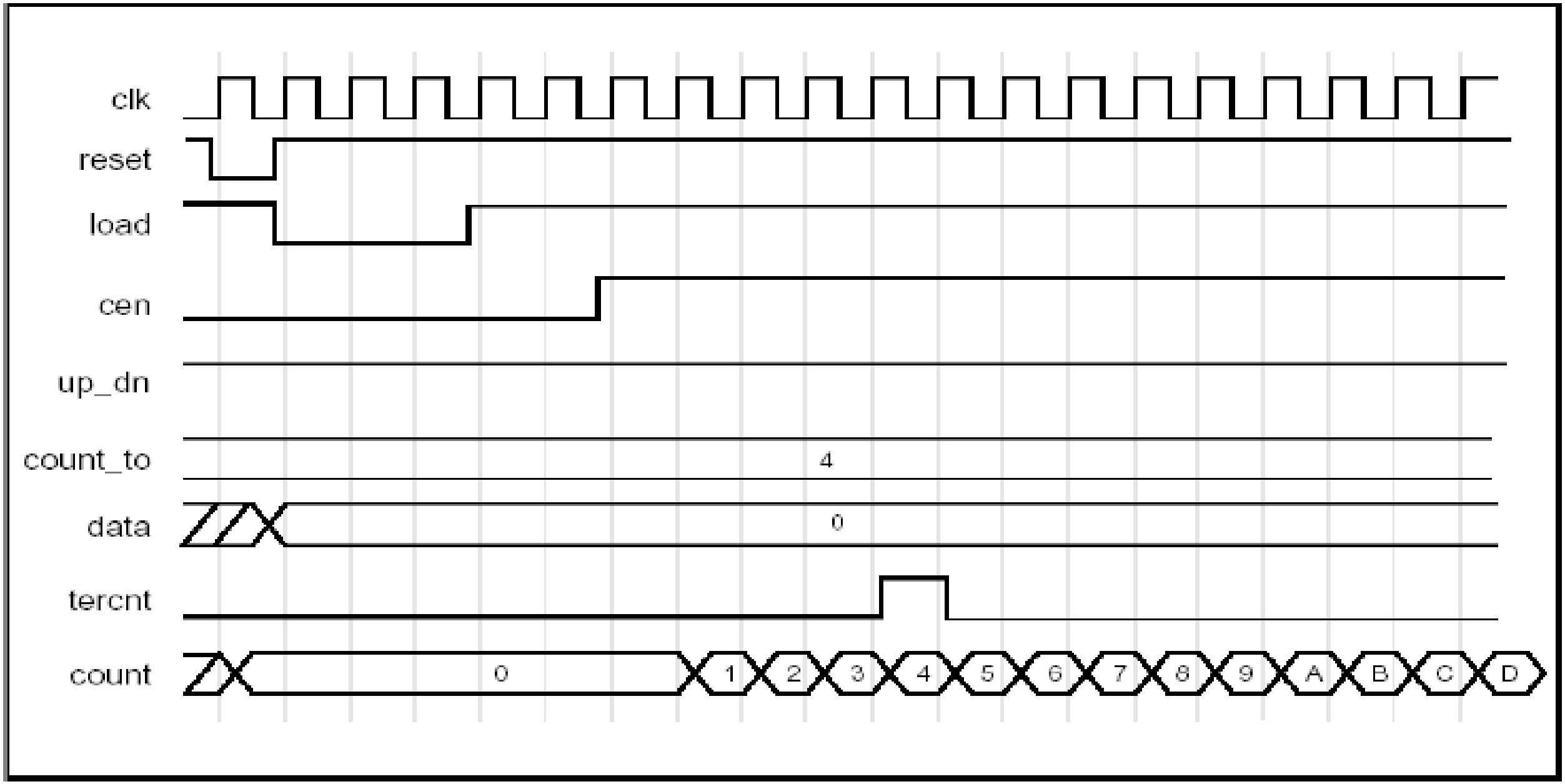
Porturile circuitului

Pin Name	Size	Type	Function
data	width	Input	Counter load input
count_to	width	Input	Count compare input
up_dn	1	Input	High for count up and low for count down
load	1	Input	Enable data load to counter, active low
cen	1	Input	Count enable, active high
clk	1	Input	Clock
reset	1	Input	Counter reset, active low
count	width	Output	Output count bus
tercnt	1	Output	Terminal count flag, active high

Tabela de adevăr

reset	load	cen	up_dn	Operation
0	X	X	X	Reset
1	0	X	X	Load
1	1	0	X	Standby
1	1	1	0	Count down
1	1	1	1	Count up

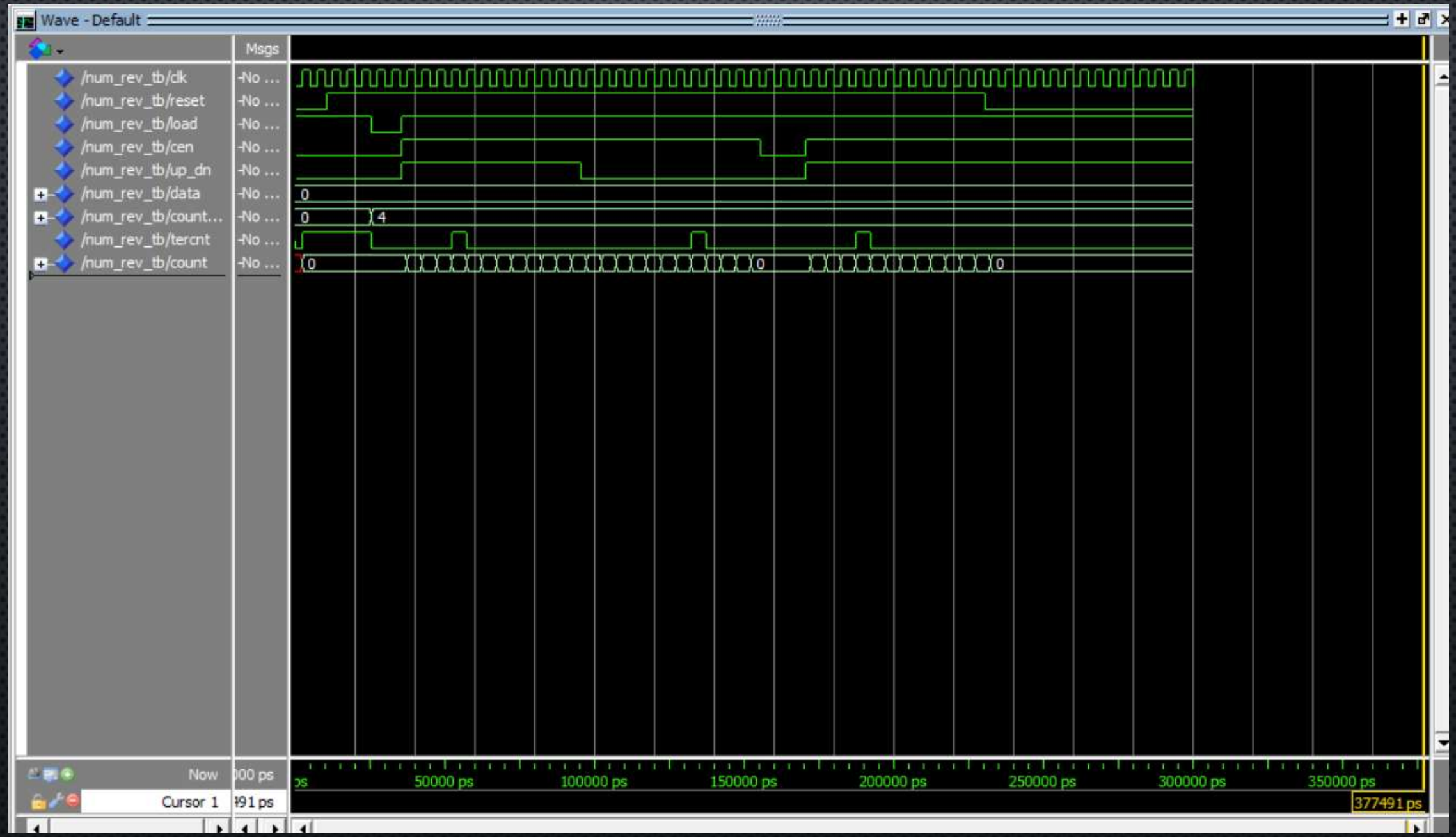
Funcționare(forme de undă)



```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity num_rev is
	Generic(
		width:integer:=4
);
	Port(
	data,count_to:in STD_LOGIC_VECTOR (width-1
downto 0);
	up_dn,load,cen,clk,reset:in STD_LOGIC;
	count:out STD_LOGIC_VECTOR (width-1 downto
0);
	tercnt:out STD_LOGIC
);
end num_rev;

architecture comp of num_rev is
	signal ci:unsigned(width-1 downto 0);

begin
	process(clk)
	begin
		if rising_edge(clk) then
			if(reset='0') then
			ci <= (others =>'0');
			else
		if(load='0') then
			ci<=unsigned(data);
		elsif(cen='1') then
			if(up_dn='1') then
				ci<=ci+1;
			else
				ci<=ci-1;
			end if;
		end if;
	end if;
	end if;
	end process;
	count<=std_logic_vector(ci);
	tercnt <= '1' when (std_logic_vector(ci)=
count_to) else '0';
end comp;
```

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity num_rev_tb is
end num_rev_tb;

architecture comp of num_rev_tb is

	constant width:integer:=4;
	constant T:time:=5 ns;
	component num_rev
	Generic(width:integer:=4);
	Port(
	data,count_to:in STD_LOGIC_VECTOR (width-1 );
downto 0);
	up_dn,load,cen,clk,reset:in STD_LOGIC;
	count:out STD_LOGIC_VECTOR (width-1
downto 0);
	tercnt:out STD_LOGIC
);
end component;

	signal clk:STD_LOGIC:='0';
	signal reset:STD_LOGIC:='0';
	signal load:STD_LOGIC:='1';
	signal cen:STD_LOGIC:='0';
	signal up_dn:STD_LOGIC:='0';
	signal data:STD_LOGIC_VECTOR(width-1
downto 0):=(others=>'0');
	signal count_to:STD_LOGIC_VECTOR(width-1
downto 0):=(others=>'0');
	signal count:STD_LOGIC_VECTOR(width-1
downto 0);
	signal tercnt:STD_LOGIC;
begin

		uut:num_rev
			Generic Map(width=>width)
			Port Map(
			clk=>clk,
			reset=>reset,
			load=>load,
			cen=>cen,
			up_dn=>up_dn,
			data=>data,
			count=>count,
			count_to=>count_to,
			tercnt=>tercnt
s_clk:process
begin
		clk<='0';
		wait for T/2;
		clk<='1';
		wait for T/2;
end process;

proces:process
begin
		reset<='0';
		wait for 2.1*T;
		reset <='1';
		wait for 3*T;
		load <='0';
		data <="0000";
		count_to <="0100";
		wait for 2*T;
		load <='1';
		cen <='1';
		up_dn <='1';
		wait for 12*T;
		up_dn <='0';
		wait for 12 *T;
		cen <='0';
		wait for 3*T;
		load <='1';
		cen <='1';
		up_dn <='1';
		wait for 12*T;
		reset <='0';
		wait for 10 ns;
		wait;
	end process;
end comp;
```

Aici putem vedea functionarea circuitului.

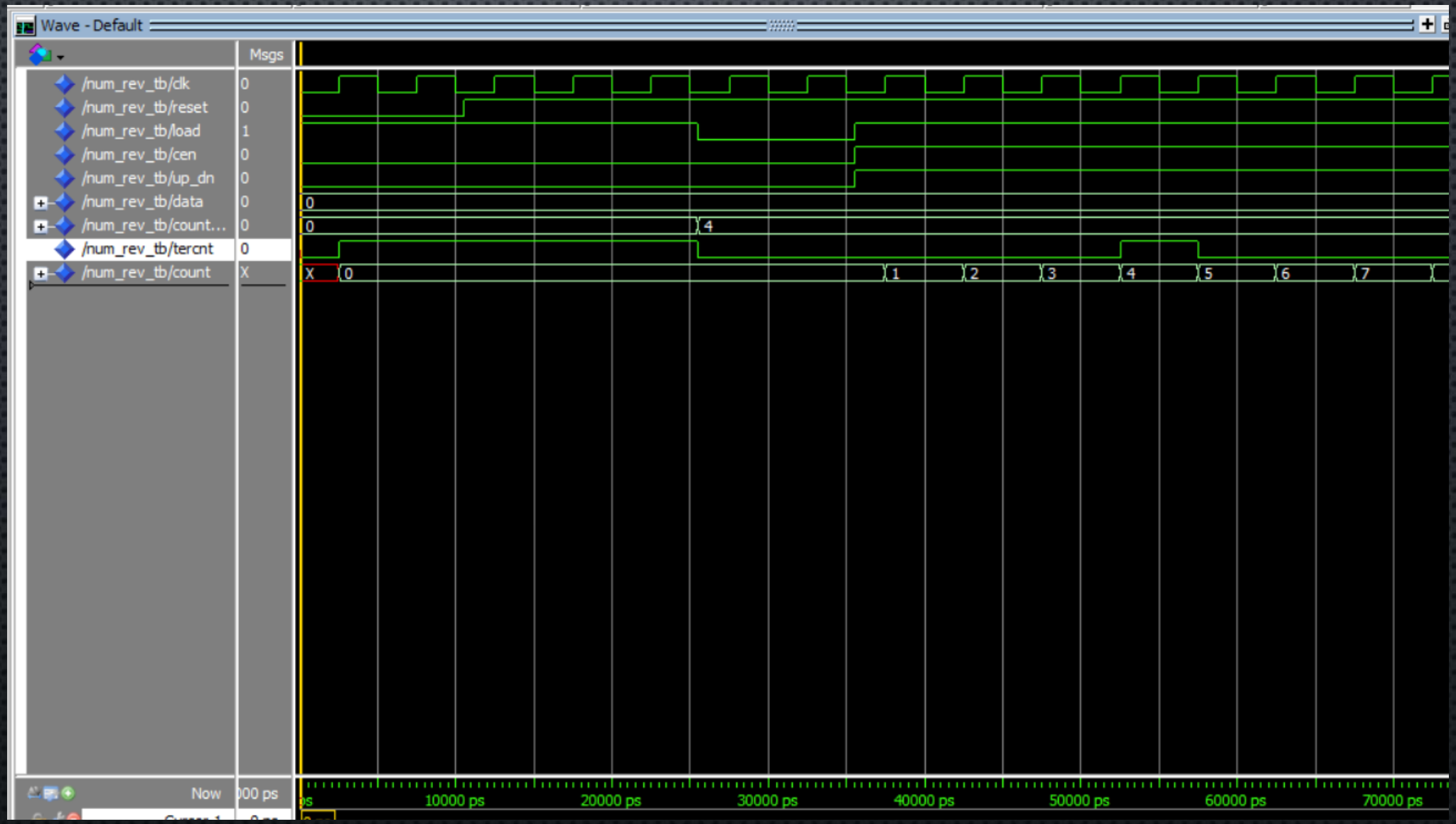


Aici am incercat sa testez o simulare cu caracteristici imprevizibile astfel incat sa vad cum functioneaza circuitul.

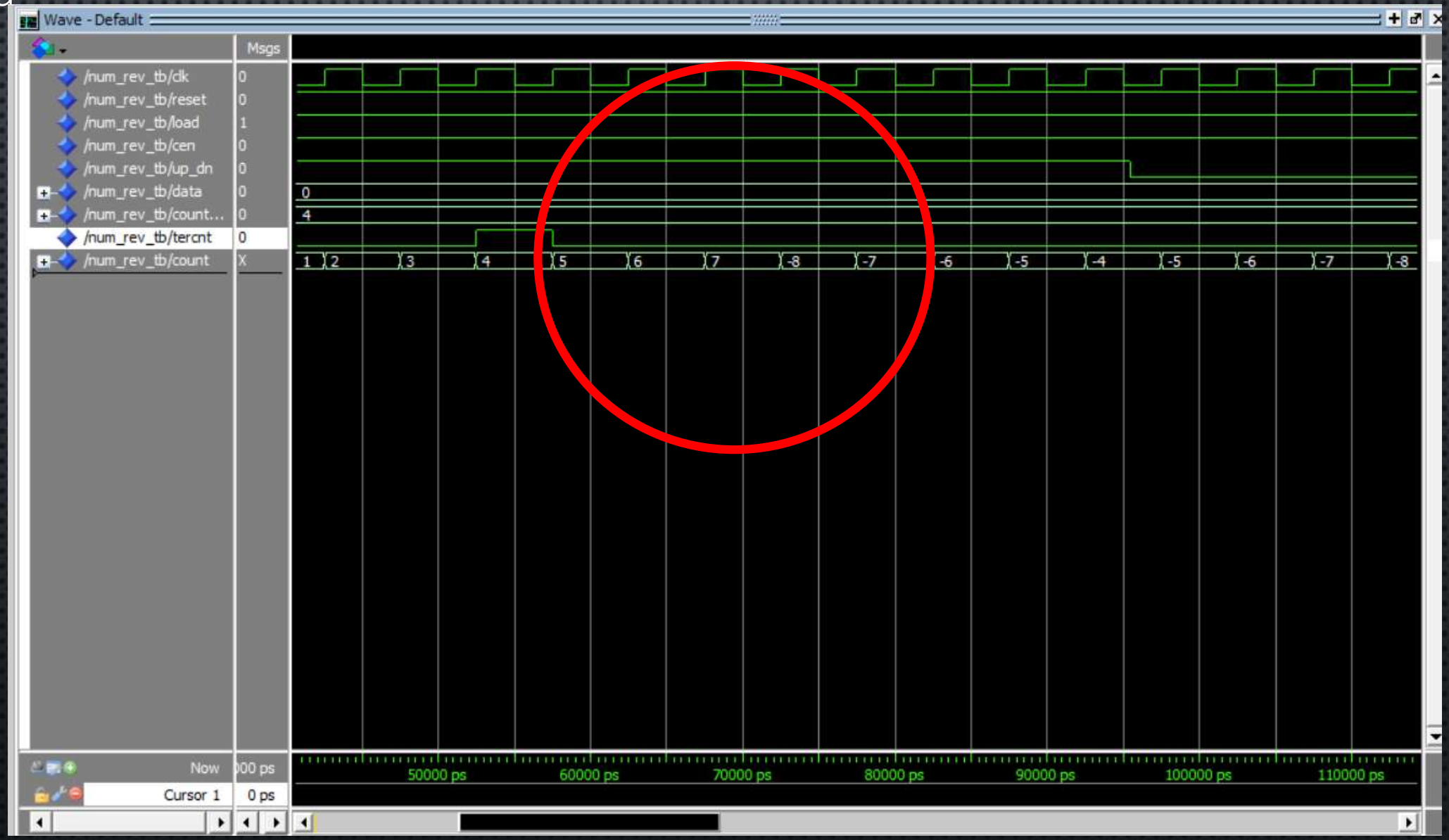
Observam count=X pana cand apare frontul crescator de ceas si ca resetul este unul sincron.

Avand in vedere ca load=0,numarul 0 sta mai mult in starea 0 exact asa cum trebuie. Numaratorul este pe 4 biti primul fiind de semn,deci count ul se realizeaza de la 7 la -8.

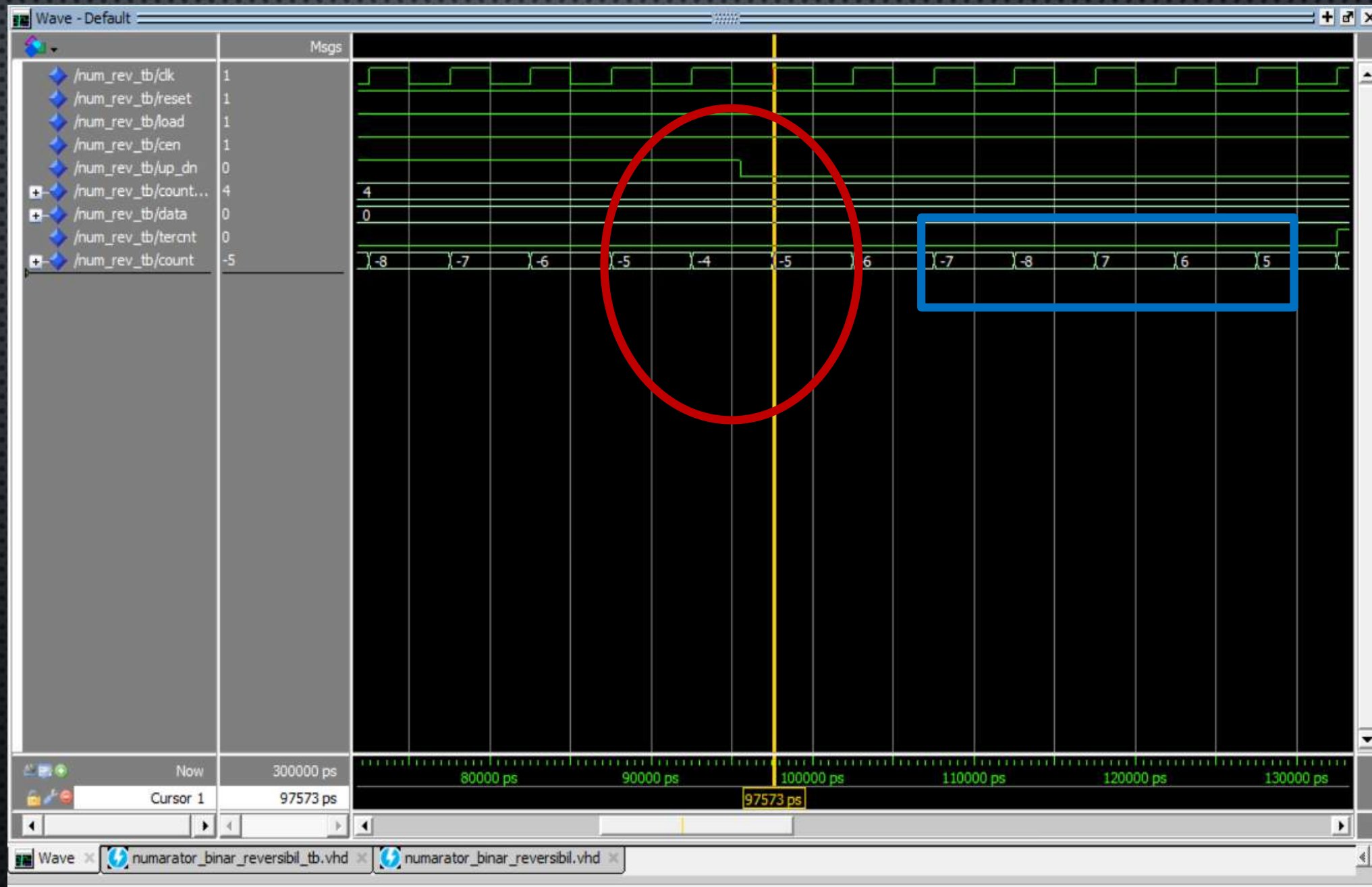
Pe langa ce am specificat mai sus putem vedea cum tercnt este pe '1' atunci cand numaratorul atinge starea 4.



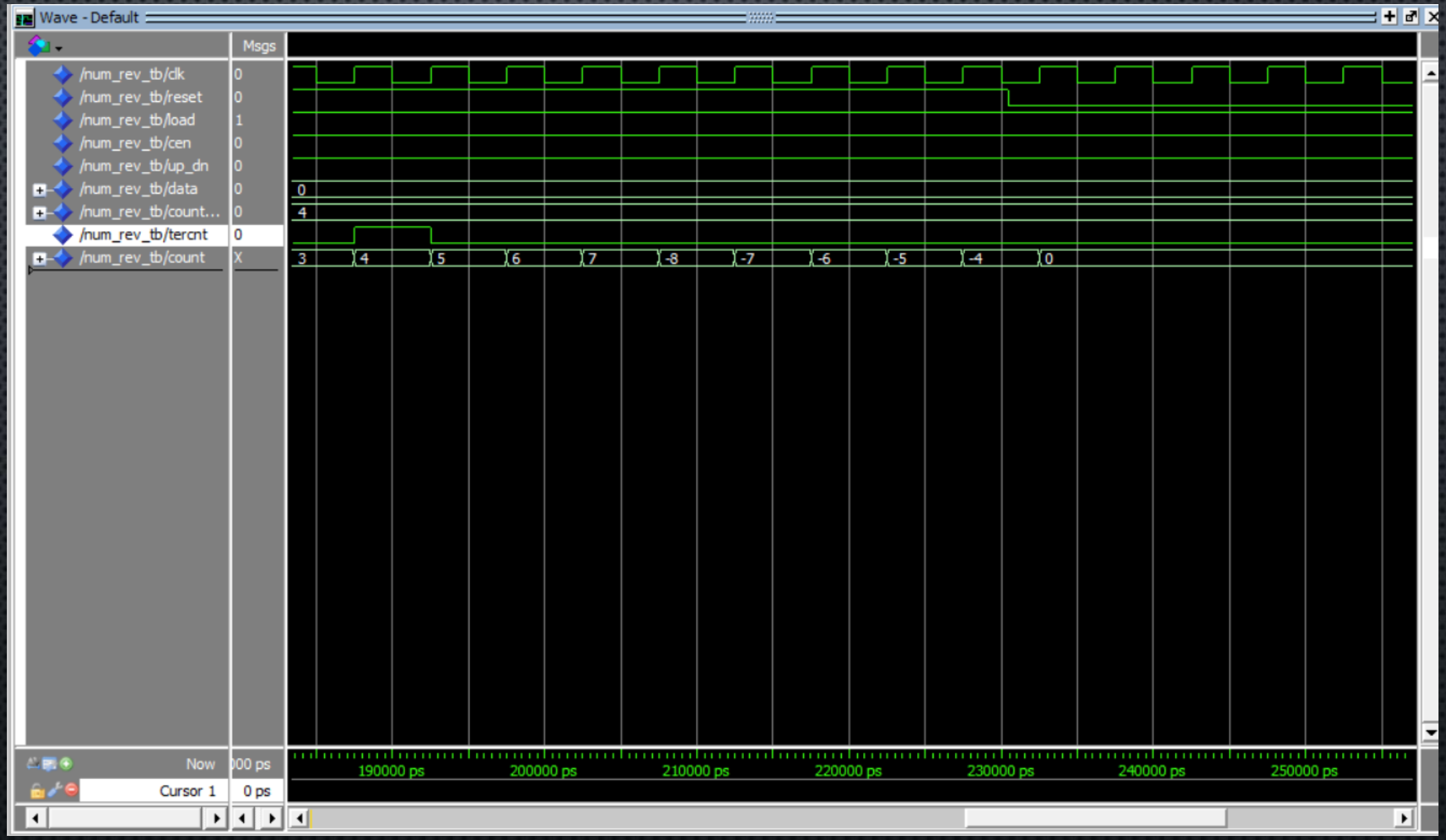
Aici am vrut sa scot in evidenta overflow-ul pe care il face numaratorul.



Aici putem observa cum la modificarea up_dn de pe starea 1 pe starea 0,la primul front pozitiv numaratorul incepe sa numere descrescator,facand din nou un overflow ulterior.



Aici inainte determinarea simularii am mai pus niste valori aleatoare astfel incat sa vad ca numaratorul functioneaza corespunzator.



VĂ MULȚUMESC!